
# Libre-V (v1.0)

## An Open-Source RISC-V SoC.

User Reference

**Version:** 1.0
**Date:** August 14, 2026
**Status:** Under Development

# About this Document

This document serves as the reference manual for the Libre-V System-on-Chip (SoC). It describes the SoC's functional blocks, their configuration options, and their intended usage. The manual is intended for software developers and hardware designers working with Libre-V platform at any abstraction.

## Release information

| Version | Date | Changes |
|---|---|---|
| 1.0 | Aug 2026 | Initial release. |

## Conventions used in this manual

- **Monospace** text indicates register names, field names, signal names, or code.
- *Italic* text indicates a value that you must supply.
- Bit ranges are written `[hi:lo]`, for example `[7:4]`.
- **Blue** panels highlight notes; **red** panels highlight warnings and cautions the user should be careful about.

## Abbreviations

| | |
|---|---|
| RF | Register File |
| RO | Read-Only Access |
| RW | Read-Write Access |
| WO | Write-Only Access (Reads are undefined/ have no meaning) |

## Notes

- The official ARM/AMBA specifications have replaced the master–slave terminology with manager–subordinate. However, some parts of the Libre-V were developed before this terminology change and the legacy master–slave nomenclature may appear in corresponding source code.
- The base implementation/specification refers to the SoC configuration that you get on cloning the repository. It is possible to configure the SoC to your needs using a Terminal User Interface. Note that this feature is still in beta testing stage and is not recommended for use without caution.

# Contents

# List of Figures

# List of Tables

# Part I

# Architecture and Software View

This part is intended for audience intending to understand the platform's architecture, configure, and adopt it in their own system.

CHAPTER 1

# Architecture

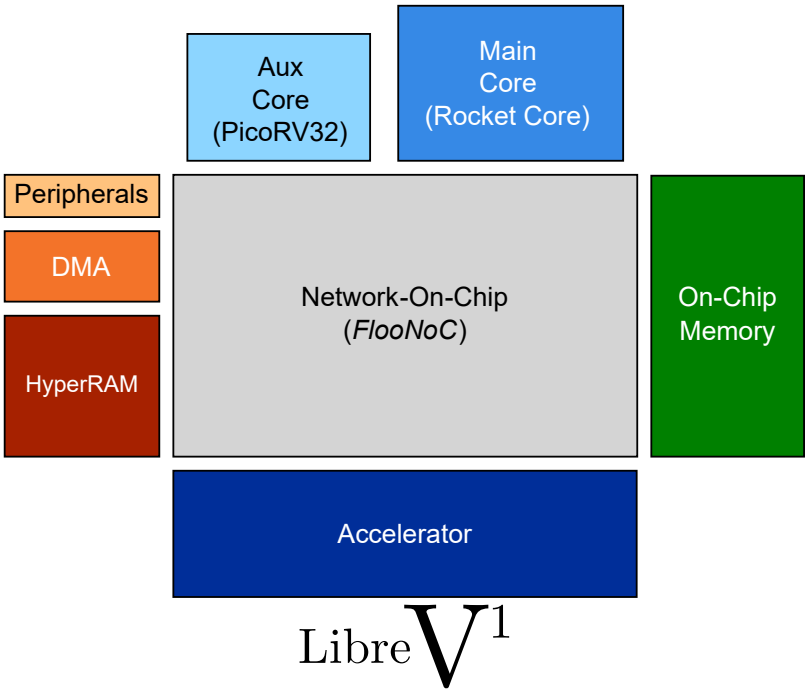



Figure 1.1: Libre-V system-on-chip concept.

## 1.1 Overview

Libre-V is a configurable system-on-chip platform intended to be used as host-CPU for testing ASICs, especially accelerators. Being composed of all open-source components, it is completely open source and licensed under the Apache-2.0 license[1]. The platform uses a 32-bit auxilary core coupled with a 64-bit main core. The auxilary core implements RV32IMC instruction set, whereas the main core implements RV64GC instruction set.

This is document does not discuss the implementation of either the RISC-V ISA nor any of the individual SoC components. For that, we direct the reader to the documentation available at respective source(s).

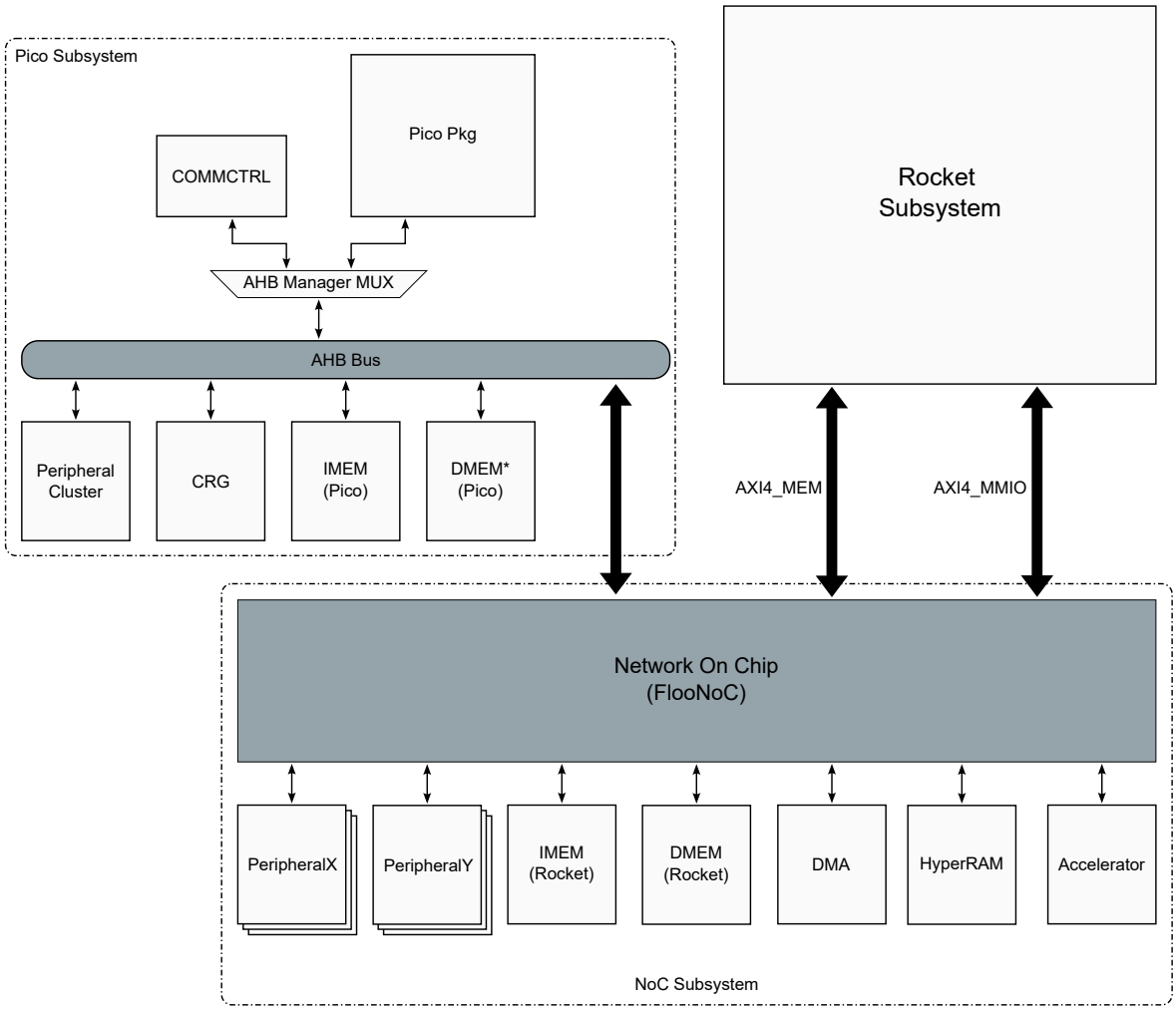



Figure 1.2: Libre-V platform block view.

## 1.2 System

Libre-V platform has 32-bit address space shared by memory-mapped I/O devices. It is composed of three subsystems:

1. Pico Subsystem
2. Rocket Subsystem
3. Network on Chip (NoC) Subsystem

See Figure 1.2 for a block-view of the platform. Note that the figure only shows two peripheral groups in the NoC subsystem. However, this is only meant for illustrations. It is possible to configure an Libre-V instance with more than 2 peripherals.

This part of the manual provides a software view of each component of the Libre-V:

- Pico Subsystem
- Rocket Subsystem
- UART
- Timer
- Libre Firmware and Device Drivers
- Libre Booting Process

[1] Individual components may be protected by their own licensing. Refer the repository for more details.

CHAPTER 2

# Pico Subsystem

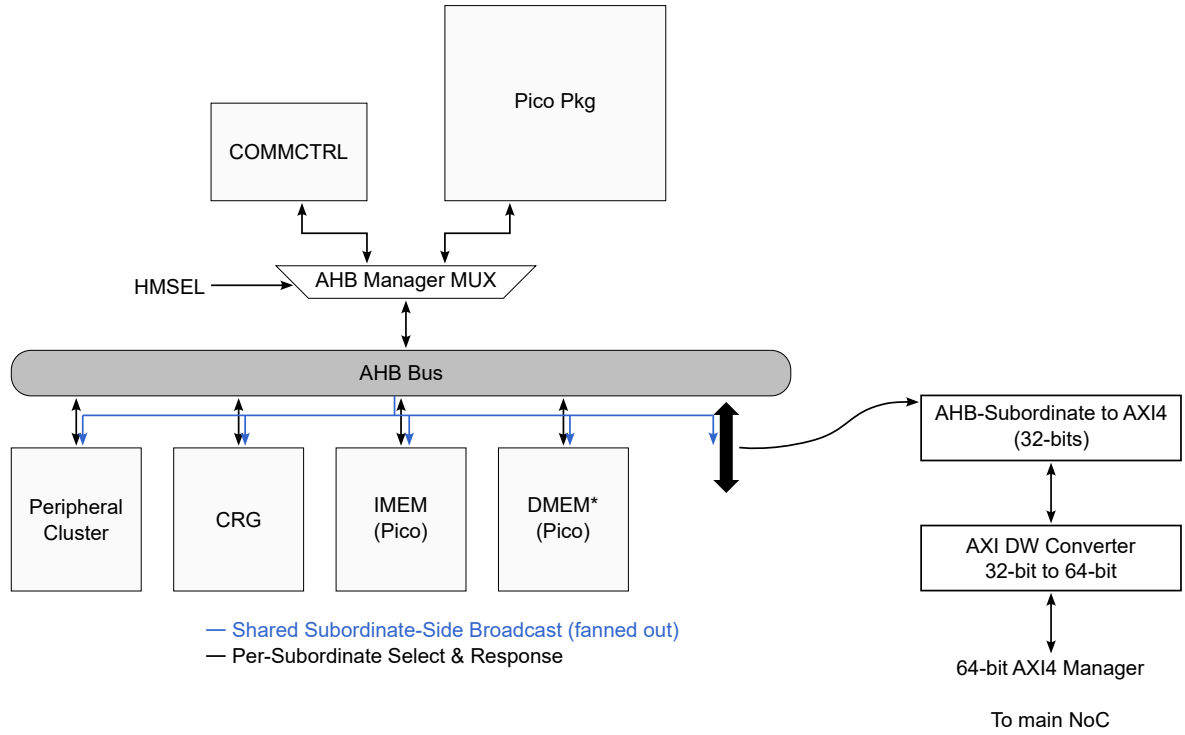



Figure 2.1: Pico subsystem block view.

The Pico subsystem is the SoC's control plane. It couples the PicoRV32 core (as Pico Package) with the host command interface, COMMCTRL, the on-chip control registers (CRG), and the Pico-local peripherals (Peripheral Cluster), all joined by an AHB Bus. It is the first block to wake at power-on and is responsible for loading and releasing the Rocket core (**??**). See Figure 2.1.

## 2.1 Pico Package

PicoRV32 core is a lightweight, auxilary core developed by Yosys. It implements RV32IMC instruction set. More generally, it can be configured as an RV32E, RV32I, RV32IC, RV32IM, RV32IMC core. It has the following features[1]:

- Small, lightweight core requiring about 750 – 2000 LUTs in 7-Series Xilinx Architecture.
- High $f_{\max}$ of about 250 – 450 MHz on Xilinx FPGAs.
- Three options for interfaces: Native Memory (`picorv32`), AXI4-Lite Manager (`picorv32_axi`), or Wishbone (`picorv32_wb`).
- A built-in interrupt controller with optional, custom ISA support.
- An optional co-processor interface.

[1] PPA results reproduced here from the repository without any additional verification.

- Configurable RF: It is possible to disable support for the registers `x16 - x32`, as well as `RDCYCLE[H]`, `RDTIME[H]`, and `RDINSTRET[H]` instructions[2]. The RF can be configured as single or dual ported.

**Relevant Link(s):**

- PicoRV32 Repository
- COMMCTRL/CHIPKIT Repository
- AMBA Bus Generator Repository

The `pico_pkg_ahb` bridges the native-memory PicoRV32 core to the subsystem as a single AHB-Lite manager. The package attaches the core's private on-chip SRAM memory and fixes the boot convention:

| Region | Base |
|---|---|
| .global, .text, .rodata, .data<br>Stack | `0x0000.0000` |

The reset vector is `0x0000.0000`, and the initial stack pointer is placed at the top of the print-buffer region (`0x0001.FC00`). The host loads Pico firmware into IMEM through COMMCTRL before handover.

FIX THIS RATIO!

## 2.2 COMMCTRL

The COMMCTRL is a UART-based communication controller. It is capable of accessing i.e., reading and writing to any address in the 32-bit address space[3]. To use this module, UART interface is exposed over which binary form of the ASCII representation of the read and/or write commands is stream.

The format of a write command is as follows:

| Byte | Field |
|---|---|
| 0 | ASCII (W or w) |
| 1 | single whitespace |
| 2 | 0 |
| 3 | ASCII(X or x) |
| 4 - 11 | WRADDR |
| 12 | single whitespace |
| 13 | 0 |
| 14 | ASCII(X/x) |
| 15 - 22 | WRDATA |
| 23 | CR |
| 24 | LF |

and a read command is formatted as:

| Byte | Field |
|---|---|
| 0 | ASCII (R or r) |
| 1 | single whitespace |
| 2 | 0 |
| 3 | ASCII(X or x) |
| 4 - 11 | RDADDR |
| 12 | CR |
| 13 | LF |

[2]Refer to the official RV32E ISA.

[3]The address should have meaning of course, else it might cause an illegal access trap somewhere in the SoC or return a garbage value.

While the write command is 25 bytes large, the read command is shorter and is zero-padded to match this length. As shown in Figure 2.1, the COMMCTRL and Pico package are multiplexed as managers of the AHB-Lite bus. The control of this AHB MUX is handled by writing to the location 0x1000.0000. Particularly, writing a 1 to this location using the COMMCTRL hands over the control to the Pico package. Writing a 0 relinquishes the control back to the COMMCTRL.

Because it is a bus master, the host can load the Pico IMEM, load the Rocket memories across the NoC, and read results back.

| Address | Symbol | Effect |
|---|---|---|
| 0x1000_0000 | COMMCTRL_HANDOVER_ADDR | Host writes 1 to hand bus ownership to the Pico. |
| 0x1000_0004 | COMMCTRL_IRQ_ADDR | Host write raises a one-cycle COMMCTRL interrupt. |

## 2.3 AHB Manager MUX and Bus

The pico_ahb_bus is the subsystem interconnect. It arbitrates two managers, COMMCTRL and the Pico core, onto one AHB fabric and decodes the address to select a subordinate. Ownership is explicit: at power-on COMMCTRL holds the bus and loads memory; when the host writes the handover address (section 2.2), ownership passes to the Pico core, which then runs its firmware. Addresses that do not fall in a local subordinate are forwarded to the NoC through the AHB-to-AXI bridge (chapter 6).

The address space 0x0000.0000 – 0x2000.0000 is reserved for the managers of the PicoSys and includes the on-chip SRAM memory of the Pico core, the control registers defined by CRG, and the peripheral cluster.

## 2.4 Clock and Reset Generator (CRG)

## 2.5 Peripheral Cluster

CHAPTER 3

# Universal Asynchronous Receiver/Transmitter (UART)

## 3.1 Overview

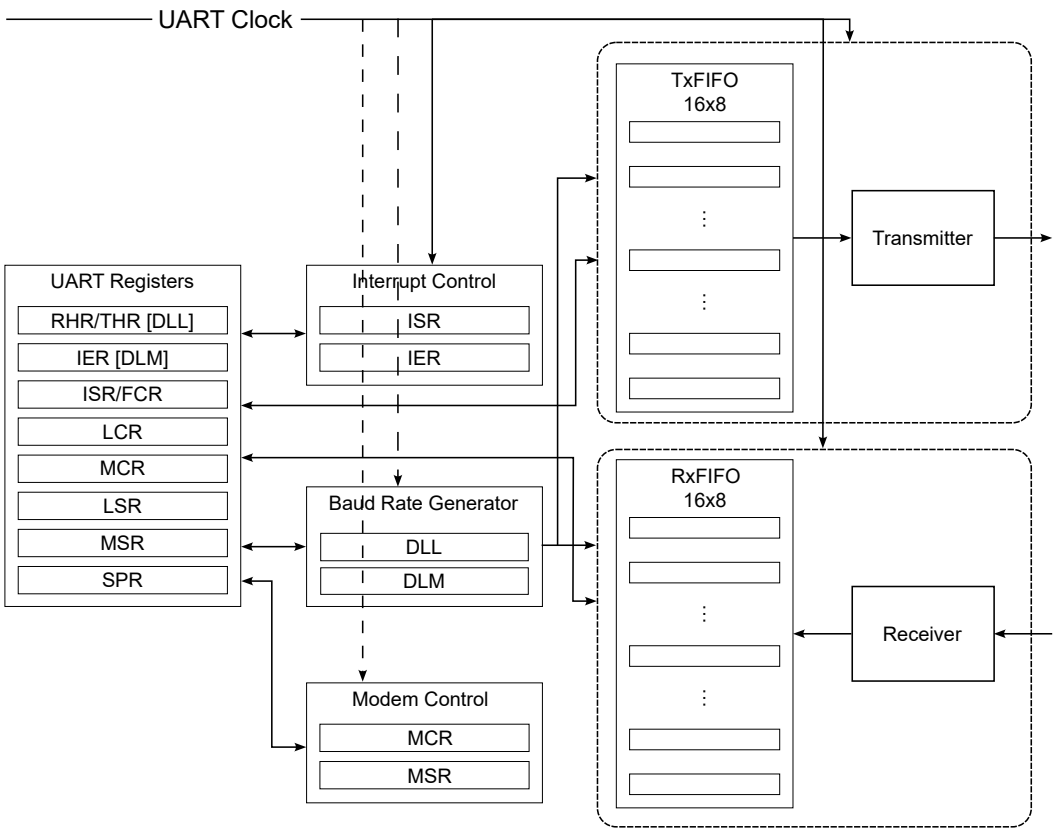



Figure 3.1: UART module high level schematic.

The configurator allows the user to add UART modules to either the peripheral cluster in the Pico subsystem or as a part of the NoC subsystem. It is connects to the AHB bus over an AHB to APB bridge that might be shared with any other peripherals in the cluster.

Fix the which UART is used for the NoC sys.

Both modules provide optional modem control pins as listed in Table 3.1. Use of most signals are implied by their names except, perhaps Ring Indicator and Carrier Detect input signals. These signals (all of them in the table above) are a part of the RS232 Interface specification. The two signals in particular are remnants of the dial-up internet/modem era, and can be safely ignored.

The modules translate the external AXI/APB interface to a register interface. Registers are byte-addressed on 4-byte boundaries. The 3-bit register index is taken from R/W address bits `addr[4:2]`:

$$\texttt{byte\_offset} = \texttt{addr[4:2]} \times 4$$

The absolute base address is defined by the SoC interconnect (for example, `0x5000.0000` in Libre-V). Offsets in this document are relative to the UART instance base.

| Signal | Function[1] |
|---|---|
| cts_n | Clear to Send (I) |
| dsr_n | Data Send Request (I) |
| ri_n | Ring Indicator (I) |
| cd_n | Carrier Detect (I) |
| rts_n | Ready to Send (O) |
| dtr_n | Data Ready (O) |
| out1_n | GP signal (optional, O) |
| out2_n | GP signal (optional, O) |

Table 3.1: Optional Modem Control Signals available in the UART module.

## 3.2 Using the UART

### 3.2.1 DLAB (Divisor Latch Access Bit)

Bit 7 of the Line Control Register (**LCR.DLAB**) multiplexes the mapping at offsets `0x00` and `0x04`:

- **DLAB = 0**: normal operation (data and interrupt enable registers).
- **DLAB = 1**: baud-rate divisor latches (**DLL**, **DLM**).

### 3.2.2 Setting the operating Baud Rate

To set the baud rate, find the baud rate divisor using:

$$\text{Divisor} = \frac{\text{SysClkFreq (in Hz)}}{16 \times \text{Baud Rate}}.$$

Having found the divisor, you need to quantize it to 16-bits[2]. The design generates train of enable pulses based of the provided clock (as opposed to actually dividing the clock signal) based on this divisor.

To configure the UART, enable the divisor latches by setting **DLAB = 1**, and set the least significant byte of the 16-bit divisor in **DLL** and the most significant byte in **DLM**.

### 3.2.3 Read/Write Shared Addresses

Several register names share the same byte offset. The hardware distinguishes them by the bus transaction direction (`a.we`):

- A **read** at offset `0x00` returns **RHR**; a **write** stores **THR** (when DLAB = 0).
- A **read** at offset `0x08` returns **ISR**; a **write** updates **FCR**.

### 3.2.4 FIFOs

When **FCR.FIFO_EN** is set, the transmit and receive paths each use a 16-entry FIFO. When cleared, only a single-byte holding register is available on each path.

[2]The UART currently only supports integer divisors.

## 3.3 Register Descriptions

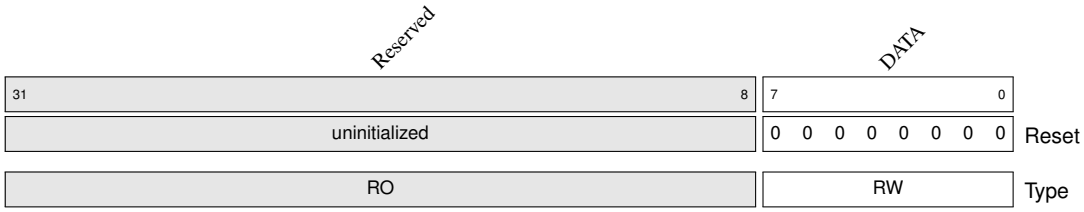
The following section describes the UART registers in the numerical order of address offset. This section will be useful for those wanting to write custom UART drivers or understand/debug existing drivers.

### Register 0: Receiver Holding Register (RHR) / Transmitter Holding Register (THR)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x000 |
| Reset value | 0x00 |
| Access type | RHR – RO; THR – WO |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Read-sensitive (RHR). DLAB must be 0.*

Register 3.1: UART DATA REGISTERS RHR/THR




**Functional Description.**

Offset `0x000` is shared by the Receiver Holding Register and the Transmitter Holding Register. A read returns the oldest received data byte from the receive FIFO (if enabled) or from the single-byte RHR. A write pushes a transmit data byte into the transmit FIFO (if enabled) or into the THR; transmission begins automatically when the transmitter is idle.

When DLAB = 1, this offset maps to the Divisor Latch LSB (**DLL**) instead; see Register 10.

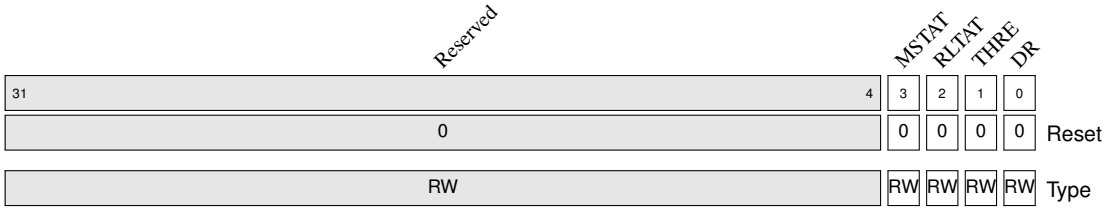
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7:0 | DATA | RHR: RO<br>THR: WO | 0 | Receive/transmit data byte. On read, returns the received character. On write, loads the next character to transmit. |

### Register 1: Interrupt Enable Register (IER)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x004 |
| Reset value | 0x00 |
| Access type | RW |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*DLAB must be 0. When DLAB = 1, this offset maps to DLM.*

Register 3.2: UART INTTERUPT ENABLE REGISTER (IER)




**Functional Description.**

The Interrupt Enable Register masks individual interrupt sources. A bit set to 1 allows the corresponding condition to contribute to the interrupt output `irq_o`. Reading or writing IER does not clear pending interrupts.

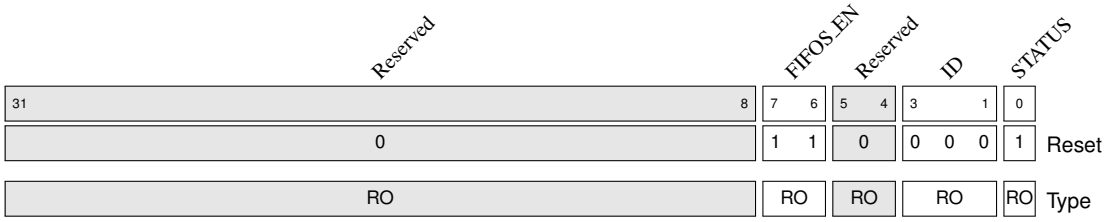
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7:4 | reserved | RO | 0 | Reserved. Read as 0. Writes are stored but have no effect. |
| 3 | MSTAT | RW | 0 | Modem Status interrupt enable. When 1, a change in modem status lines may assert an interrupt. |
| 2 | RLSTAT | RW | 0 | Receive Line Status interrupt enable. When 1, overrun, parity, framing, or break errors may assert an interrupt. |
| 1 | THRE | RW | 0 | Transmitter Holding Register Empty interrupt enable. When 1, an empty THR/TX FIFO may assert an interrupt. |
| 0 | DR | RW | 0 | Data Ready / Reception Timeout interrupt enable. When 1, received data available or a character timeout (FIFO mode) may assert an interrupt. |

## Register 2: Interrupt Status Register (ISR)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x008 |
| Reset value | 0xC1 |
| Access type | RO |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Read-only. DLAB must be 0.*

Register 3.3: UART INTERRUPT STATUS REGISTER (ISR)




**Functional Description.**

The Interrupt Status Register reports whether an interrupt is pending and, if so, the highest priority interrupt source. The register is implemented in `obi_uart_interrupts.sv` and is not stored in the main register flip-flop array. Reading ISR identifies the interrupt source but does not by itself clear all interrupt conditions; associated status registers (LSR, MSR, RHR) may need to be serviced.

| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7:6 | FIFOS_EN | RO | 2'b11 | FIFO capability field. Hardwired to `2'b11` to indicate 16550A-style FIFO support. |
| 5:4 | reserved | RO | 0 | Reserved. Read as 0. |
| 3:1 | ID | RO | 0 | Interrupt identification code when **STATUS** = 0:<br>• `011`: Receive line status<br>• `010`: Received data available<br>• `110`: Character timeout (FIFO mode)<br>• `001`: THR empty<br>• `000`: Modem status |
| 0 | STATUS | RO | 1 | Interrupt status. 0 = interrupt pending; 1 = no interrupt pending. |

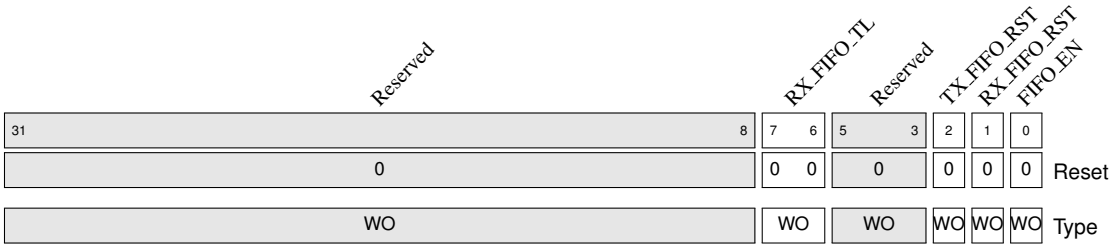
The ID field is updated in the priority encoder in the interrupt submodule.

## Register 3: FIFO Control Register (FCR)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x008 |
| Reset value | 0x00 |
| Access type | WO |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Write-only. Shares offset with ISR.*

Register 3.4: UART FIFO CONTROL REGISTER (FCR)




**Functional Description.**

The FIFO Control Register configures the on-chip transmit and receive FIFOs that are a part of the UART Tx and Rx submodules. Writes to this register at offset `0x008` update FCR; reads at the same offset return ISR.

The RX and TX FIFOs are each 16 entries deep. The RX FIFO stores 11 bits per entry (8 data bits plus break, framing, and parity error flags). The TX FIFO stores 8-bit characters.

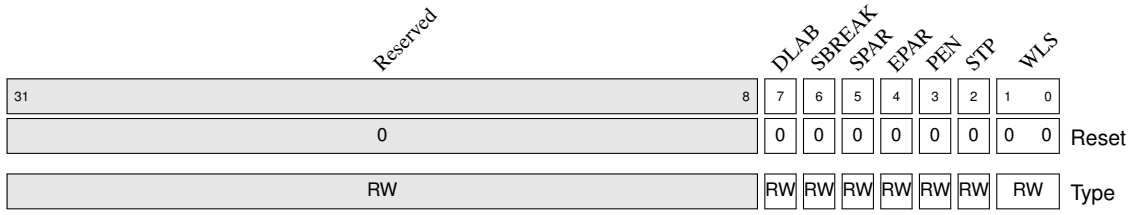
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7:6 | RX_FIFO_TL | WO | 0 | Receive FIFO trigger level:<br>• `00`: 1 character<br>• `01`: 4 characters<br>• `10`: 8 characters<br>• `11`: 14 characters |
| 5:3 | reserved | WO | 0 | Reserved. |
| 2 | TX_FIFO_RST | WO | 0 | Write 1 to reset/clear the transmit FIFO. |
| 1 | RX_FIFO_RST | WO | 0 | Write 1 to reset/clear the receive FIFO. |
| 0 | FIFO_EN | WO | 0 | FIFO enable. 0 = 8250-style single-byte THR/RHR; 1 = 16550A 16-byte FIFO mode. |

## Register 4: Line Control Register (LCR)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x00C |
| Reset value | 0x00 |
| Access type | RW |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Accessible regardless of DLAB.*

Register 3.5: UART LINE CONTROL REGISTER (LCR)




**Functional Description.**

The Line Control Register defines the serial frame format and the DLAB address-multiplexing bit. Changes to word length, parity, and stop bits take effect for subsequently transmitted and received characters.

| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7 | DLAB | RW | 0 | Divisor Latch Access Bit. When 1, offsets `0x00` and `0x04` access DLL and DLM. When 0, they access RHR/THR and IER. |
| 6 | SBREAK | RW | 0 | Set Break. When 1, forces the transmit output to a spacing (low) condition. |
| 5 | SPAR | RW | 0 | Stick Parity. When parity is enabled, forces parity bit to 0 (if EPAR = 1) or 1 (if EPAR = 0). |
| 4 | EPAR | RW | 0 | Even Parity select. 0 = odd parity; 1 = even parity (when parity enabled). |
| 3 | PEN | RW | 0 | Parity Enable. When 1, a parity bit is transmitted and checked. |
| 2 | STP | RW | 0 | Stop Bits. 0 = one stop bit; 1 = two stop bits (1.5 stop bits when word length is 5 bits). |

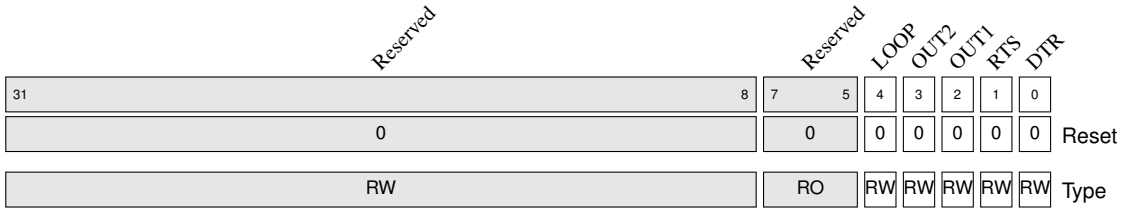
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 1:0 | WLS | RW | 0 | Word Length Select:<br>• `00`: 5 data bits<br>• `01`: 6 data bits<br>• `10`: 7 data bits<br>• `11`: 8 data bits |

## Register 5: Modem Control Register (MCR)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x010 |
| Reset value | 0x00 |
| Access type | RW |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Accessible regardless of DLAB.*

Register 3.6: UART MODEM CONTROL REGISTER (MCR)




**Functional Description.**

The Modem Control Register drives modem output pins and enables internal loopback for test. Modem outputs are active-low on the external pins (`rts_no`, `dtr_no`, etc.).

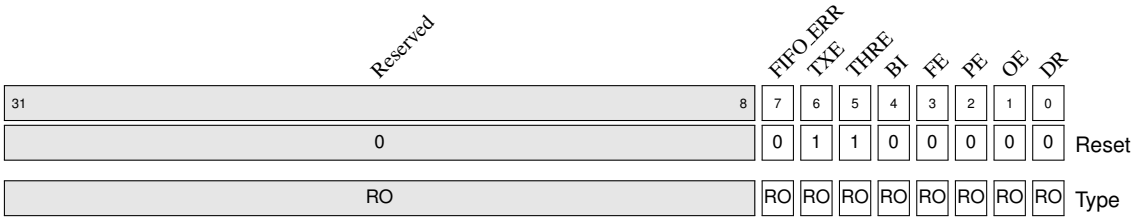
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7:5 | reserved | RO | 0 | Reserved. Read as 0. |
| 4 | LOOP | RW | 0 | Loopback. When 1, the transmitter output is looped back to the receiver input internally. |
| 3 | OUT2 | RW | 0 | Auxiliary output 2 (optional GPIO). |
| 2 | OUT1 | RW | 0 | Auxiliary output 1 (optional GPIO). |
| 1 | RTS | RW | 0 | Request To Send output. |
| 0 | DTR | RW | 0 | Data Terminal Ready output. |

## Register 6: Line Status Register (LSR)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x014 |
| Reset value | 0x60 |
| Access type | RO |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Read-sensitive. DLAB must be 0.*

Register 3.7: UART LINE STATUS REGISTER (LSR)




**Functional Description.**

The Line Status Register provides transmit readiness and receive status, including error flags. Unlike some UART designs, error information is *not* embedded in the RHR read data; errors are reported only in LSR. Reading LSR clears the receive-line-status interrupt. Individual error bits are cleared on LSR read when no additional FIFO errors remain.

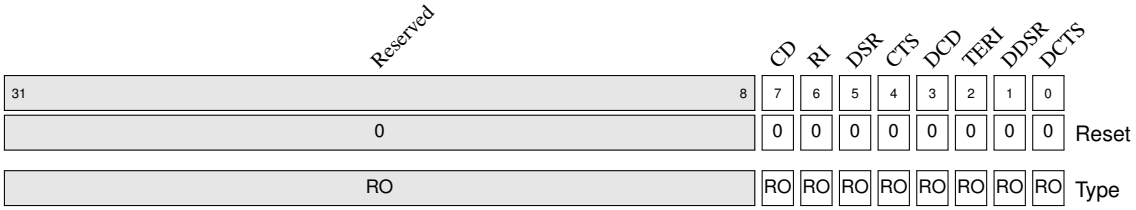
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7 | FIFO_ERR | RO | 0 | FIFO Error. Set when at least one character with an error was received into the RX FIFO. |
| 6 | TXE | RO | 1 | Transmitter Empty. 1 when the transmitter shift register and holding logic are completely idle. |
| 5 | THRE | RO | 1 | THR Empty. 1 when the THR or TX FIFO can accept a new write. |
| 4 | BI | RO | 0 | Break Indicator. 1 when a break condition was detected on the receive line. |
| 3 | FE | RO | 0 | Framing Error. 1 when the stop bit was not received as expected. |
| 2 | PE | RO | 0 | Parity Error. 1 when the received parity bit did not match. |
| 1 | OE | RO | 0 | Overrun Error. 1 when a new character arrived before the previous one was read (RHR mode) or when the RX FIFO was full (FIFO mode). |
| 0 | DR | RO | 0 | Data Ready. 1 when received data is available in RHR or the RX FIFO. |

## Register 7: Modem Status Register (MSR)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x018 |
| Reset value | 0x00 |
| Access type | RO |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Read-sensitive. DLAB must be 0.*

Register 3.8: UART MODEM STATUS REGISTER (MSR)




**Functional Description.**

The Modem Status Register reports the state of modem input signals and edge-detected changes. Delta bits (bits 4–7 in this implementation) are sticky and remain set until MSR is read.

Modem input pins are active-low externally; the register stores the logical (non-inverted) levels and change indicators.

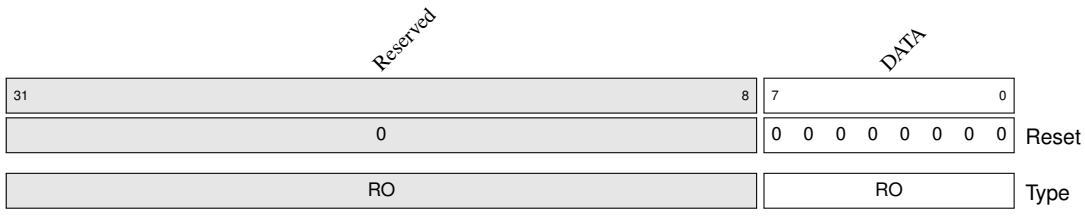
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7 | CD | RO | 0 | Carrier Detect input level. |
| 6 | RI | RO | 0 | Ring Indicator input level. |
| 5 | DSR | RO | 0 | Data Set Ready input level. |
| 4 | CTS | RO | 0 | Clear To Send input level. |
| 3 | DCD | RO | 0 | Delta Carrier Detect. Set when CD input changes state. |
| 2 | TERI | RO | 0 | Trailing Edge Ring Indicator. Set on high-to-low transition of RI. |
| 1 | DDSR | RO | 0 | Delta Data Set Ready. Set when DSR input changes state. |
| 0 | DCTS | RO | 0 | Delta Clear To Send. Set when CTS input changes state. |

## Register 8: Scratch Pad Register (SPR)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x01C |
| Reset value | 0x00 |
| Access type | RO |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Not implemented.*

Register 3.9: UART SCRATCH PAD REGISTER (SPR)




**Functional Description.**

The optional Scratch Pad Register is defined by the 16550A standard as a software-writable storage location. In this implementation, reads always return `0x00` and writes are silently ignored. No error is generated.

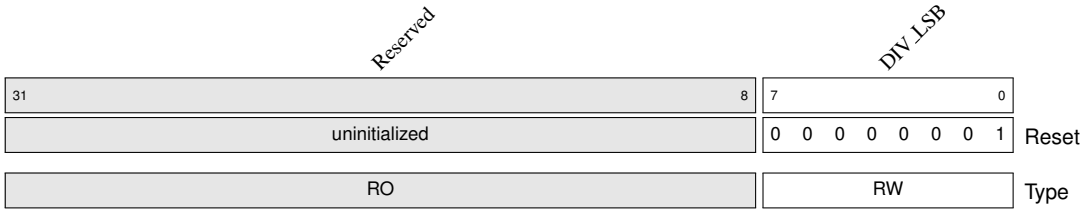
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7:0 | DATA | RO | 0 | Always reads zero. Writes have no effect. |

## Register 9: Divisor Latch LSB (DLL)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x000 |
| Reset value | 0x01 |
| Access type | RW |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*DLAB must be 1. Shares offset with RHR/THR.*

Register 3.10: UART DIVISOR LATCH LSB (DLL)




**Functional Description.**

When DLAB = 1, offset `0x000` maps to the least significant byte of the 16-bit baud rate divisor. The divisor is combined with DLM as:

A divisor of 0 is invalid and disables baud-rate generation. Writing DLL or DLM resets the internal baud counters.

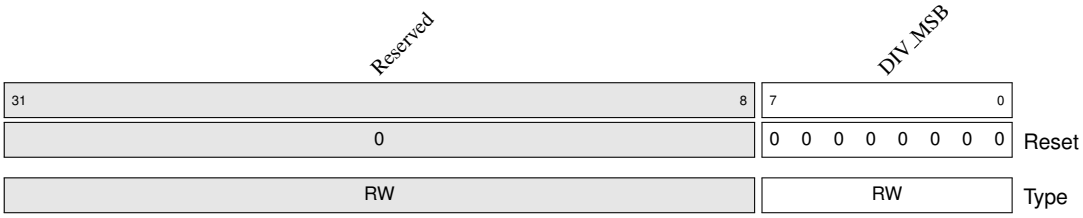
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7:0 | DIV_LSB | RW | 0x01 | Least significant byte of the baud divisor latch. |

## Register 10: Divisor Latch MSB (DLM)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x004 |
| Reset value | 0x00 |
| Access type | RW |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*DLAB must be 1. Shares offset with IER.*

Register 3.11: UART DIVISOR LATCH MSB (DLM)




**Functional Description.**

When DLAB = 1, offset `0x004` maps to the most significant byte of the baud rate divisor. See Register 9 for the baud-rate formula.

| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 7:0 | DIV_MSB | RW | 0x00 | Most significant byte of the baud divisor latch. |

### 3.3.1 Address Map Summary

| Offset | DLAB = 0 (read) | DLAB = 0 (write) | DLAB = 1 (read) | DLAB = 1 (write) |
|---|---|---|---|---|
| 0x000 | RHR | THR | DLL | DLL |
| 0x004 | IER | IER | DLM | DLM |
| 0x008 | ISR | FCR | ISR | FCR |
| 0x00C | LCR | LCR | LCR | LCR |
| 0x010 | MCR | MCR | MCR | MCR |
| 0x014 | LSR | — | — | — |
| 0x018 | MSR | — | — | — |
| 0x01C | SPR | SPR | SPR | SPR |

CHAPTER 4

# Timer Family

Libre-V offers three different timer options:

1. Basic Timer
2. General Purpose Timer
3. Advanced Timer/PWM

These timer variants differ in their functionality. The feature set increases in the order listed above. All modules are memory mapped APB peripherals.

## 4.1 Basic Timer (BTimer)

The BTimer module is the "APB timer" from the PULP platform. It is a free-running 32-bit up-counter with a programmable clock prescaler and a compare register, and it raises two physical[1] interrupts: a counter *overflow* and a *compare match*.

Note that the BTimer module can instantiate several `timer` cores behind an internal APB address mux (selected by `PADDR`). See **??** for a high level view of the module.

The Btimer core exposes three 32-bit registers. They are byte-addressed on 4-byte boundaries; the register index is taken from APB address bits `PADDR[3:2]`:

$$\texttt{byte\_offset} = \texttt{PADDR[3:2]} \times 4$$

> The base address of a Btimer can be found in the generated software library file(s) as these depend on the user-specific configuration. Offsets in this document are relative to the timer instance base.

# Using the Btimer

Figure Figure 4.1 illustrates the control flow of the Btimer module.

### 4.1.1 Counting and the Prescaler

When **CTRL.EN** is set, the counter advances at a rate governed by the 3-bit prescaler field **CTRL.PSC**. A free-running *cycle counter* counts input clock cycles from `0` up to the prescaler value and then wraps:

- **PSC = 0** (normal mode): the counter increments on every clock cycle.
- **PSC** $\neq$ **0** (prescaled mode): the counter increments once every $(\mathrm{PSC}+1)$ clock cycles.

So the effective counting rate is $f_{\mathrm{clk}}/(\mathrm{PSC}+1)$.

[1] By physical we mean that the interrupts are exposed as output ports as opposed to updating registers. These interrupts are made software-readable by connecting it to interrupt ports of the cores either pico or rocket.

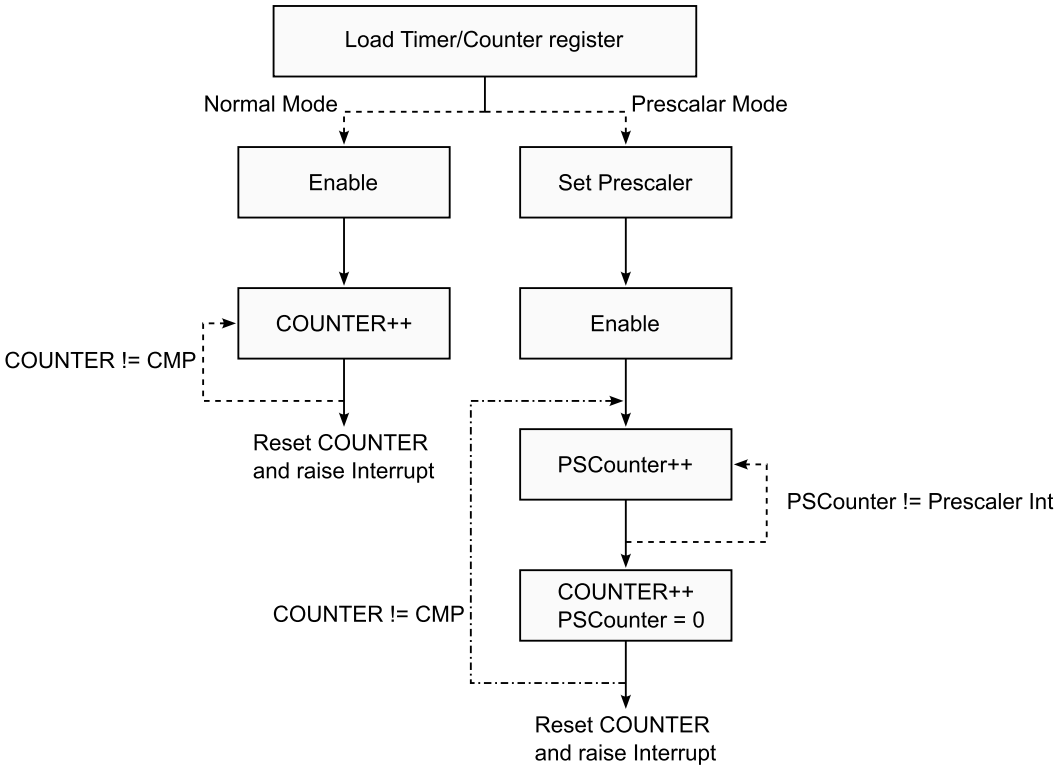



Figure 4.1: Btimer control flow

### 4.1.2 Compare, Overflow, and Auto-reset

The counter is compared against the **CMP** register on each tick:

- **Compare match**: when `CMP` $\neq 0$ and the counter equals `CMP`, the compare-match interrupt (`irq_o[1]`) is asserted and the counter is reset to `0`. A `CMP` value of `0` disables the compare function.
- **Overflow**: when the counter reaches `0xFFFF_FFFF` and the next tick occurs, the overflow interrupt (`irq_o[0]`) is asserted and the counter wraps to `0`.

Both interrupts are single-cycle pulses generated combinationally on the tick. In addition, *writing* the `CMP` register resets the counter to `0`, which makes it easy to start a fresh timed interval. This gives a straightforward periodic-tick usage: program a prescaler and a compare value, enable the timer, and take a compare-match interrupt every $(\mathrm{CMP}+1)\times(\mathrm{PSC}+1)$ clock cycles.

### 4.1.3 Register Descriptions

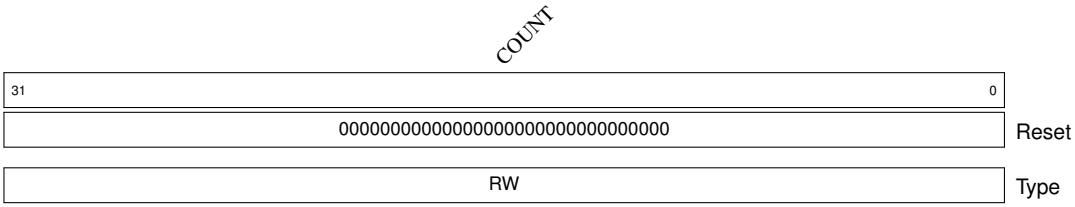
The following section describes the TIMER registers in the numerical order of address offset.

## Register 0: Timer Counter Register (TIMER)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x000 |
| Reset value | 0x00 |
| Access type | RW |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Free-running count. Auto-resets on match/overflow/CMP write.*

Register 4.1: TIMER COUNTER REGISTER (TIMER)




**Functional Description.**

Offset `0x000` holds the current 32-bit counter value. A read returns the instantaneous count. A write preloads the counter, allowing software to start from an arbitrary value. When the timer is enabled the hardware advances this register at the prescaled rate, and clears it to `0` on a compare match, on overflow, or when the **CMP** register is written.

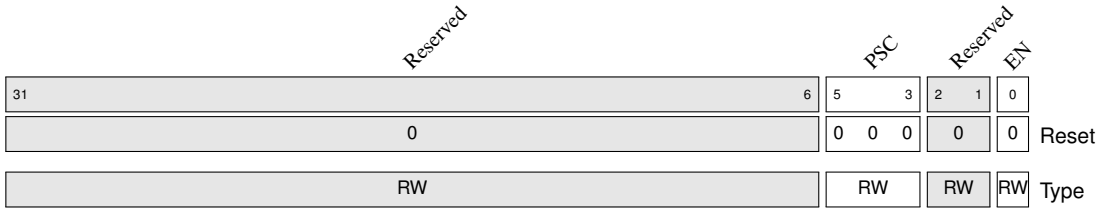
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 31:0 | COUNT | RW | 0 | Current timer count. Readable at any time; a write preloads the counter. Advances at $f_{\mathrm{clk}}/(\mathrm{PSC}+1)$ while **CTRL.EN** = 1. |

## Register 1: Timer Control Register (CTRL)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x004 |
| Reset value | 0x00 |
| Access type | RW |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Only EN (bit 0) and PSC (bits 5:3) are functional.*

Register 4.2: TIMER CONTROL REGISTER (CTRL)




**Functional Description.**

The Control Register enables the counter and sets the prescaler. The full 32-bit word is stored, so the reserved bits are writable and read back their last written value, but they have no effect on operation.

| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 31:6 | reserved | RW | 0 | Reserved. Stored but have no effect. |

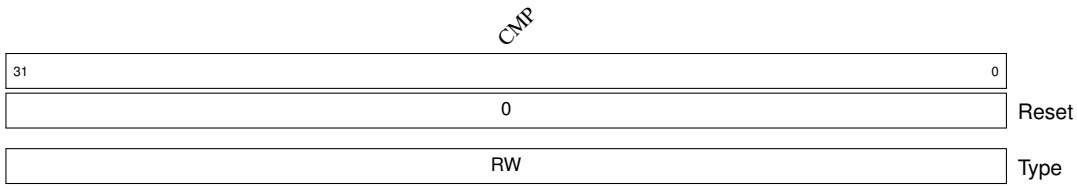
| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 5:3 | PSC | RW | 0 | Prescaler. The counter increments once every $(\mathrm{PSC}+1)$ clock cycles. `PSC = 0` selects normal mode (increment every cycle). |
| 2:1 | reserved | RW | 0 | Reserved. Stored but have no effect. |
| 0 | EN | RW | 0 | Enable. When 1, the counter runs; when 0, the counter is frozen at its current value. |

## Register 2: Timer Compare Register (CMP)

| Property | Value |
|---|---|
| Base address | SoC-defined |
| Byte offset | 0x008 |
| Reset value | 0x00 |
| Access type | RW |
| Bus data width | 32-bit (only bits [7:0] are used) |
| Register width | 8 bits |

*Writing CMP resets the counter to 0. CMP = 0 disables compare.*

Register 4.3: TIMER COMPARE REGISTER (CMP)




**Functional Description.**

Offset `0x008` holds the 32-bit compare value. When the counter reaches this value (and the value is non-zero), the compare-match interrupt `irq_o[1]` is asserted and the counter is reset to `0`, producing a periodic tick of period $(\mathrm{CMP}+1)\times(\mathrm{PSC}+1)$ clock cycles. Writing this register also immediately resets the counter to `0`, so software should program **CMP** before starting the count. A value of `0` disables the compare path, leaving only the overflow interrupt active.

| Bit(s) | Name | Type | Reset | Description |
|---|---|---|---|---|
| 31:0 | CMP | RW | 0 | Compare value. On `COUNT == CMP` (with `CMP` $\neq 0$) the compare-match interrupt fires and the counter clears. Writing this field also clears the counter. `CMP = 0` disables the compare match. |

### 4.1.4 Interrupts

The timer core drives a 2-bit interrupt output `irq_o[1:0]`:

| Bit | Name | Condition |
|---|---|---|
| 0 | Overflow | Counter wraps from `0xFFFF_FFFF`. |
| 1 | Compare match | Counter equals a non-zero `CMP`. |

### 4.1.5 Address Map Summary

| Offset | Register | Access |
|---|---|---|
| 0x000 | TIMER (counter) | RW |
| 0x004 | CTRL (control) | RW |
| 0x008 | CMP (compare) | RW |

CHAPTER 5

# Libre Software and Control

## 5.1 PicoRV32 Interrupt Handler

The PicoRv32 core has a built-in controller capable of handling 32 sources. The sources should be wired into the `irq` input port of the core/package. The handler also controles the `eoi` (end of interrupt) signals, which go high for the handled interrupt when the specific interrupt handler is called.

IRQs 0-2 are reserved for internal sources. These include:

1. IRQ 0 - Timer interrupt
2. IRQ 1 - EBREAK/ECALL or Illegal INstructions
3. IRQ 2 - Bus error due to unaligned memory access.

4 additionals 32-bit regsiters (`q0-3`) are provided for IRQ handling. WHen the IRQ handler is called, the register `q0` contains the return address and `q1` contains the bitmask of all IRQs to be handled. This means one call to the interrupt handler needs to service more than one IRQ when more than one bit is set in `q1`. When compressed instruction mode is enabled, then the LSB of `q0` is set when the interrupt instruction is a compressed instruction. This can be userd if the IRQ handler wants to decode the interrupted instruciton. The other additional reisters are uninitializaed and scratch registers.

Custom instructions defined for interrupt handling:

1. `get rd, qs`
   Copy the contents from a q-register to a general-purpose register.
2. `retirq`
   Return from interrrupt. As side effects, the contents of `q0` are copied to the program counter and interrupts are re-enabled.
3. `maskirq`:
   There is an IRQ MAsk register containing a birmask of masked interrupts. This instruction writes a new valie to the ieq mask register and reads the old value.
4. `waitirq`
   Pause execution until an interrupt becomes pending.
5. `timer`
   Reset the timer counter to a new value. The counter counts down clock cycles and triggers the timer interrupt when transitioning from 1 to 0. Setting the counter to zero disables the timer. The old value of the counter is written to rd.

# Part II

# SoC Integration

This part is intended for audience intending to understand the platform's architecture and modify/develop their own version at an RTL level. Libre platform is composed of three subsystems:

- Pico Subsystem
- Rocket Subsystem
- Network-on-Chip (NoC) Subsystem

CHAPTER 6

# Pico Subsystem

The Pico subsystem (picosys) is a auxiliary subsystem organized around an AHB-Lite bus. It is used to control the rest of the SoC. The AHB-Lite bus is driven by two managers: COMMCTRL/UART Controller and PicoRV32 core package (picopkg). However, both these managers are time-multiplexed using an AHB MUX, meaning that at any time, there is only a single manager driving the bus. For additional control and interaction, the bus supports a configurable peripheral bus with options like UART, Timers, etc., on-chip SRAM memory, clock and reset generator (CRG), and a bridge to connect to the AXI port of the NoC.

Picosys uses the CRG to bring up and control the remainder of the SoC. In this chapter, we breakdown the construction and integration of the picosys in Libre-V platform:

- PicoRV32 Core and Package
- COMMCTRL
- AHB-Lite MUX and Bus
- Clock and Reset Generator (CRG)
- Peripheral Cluster
- On-chip SRAM
- AHB2AXI adapter

## 6.1 PicoRV32 Core and Package

CHAPTER 7

# On Chip SRAM

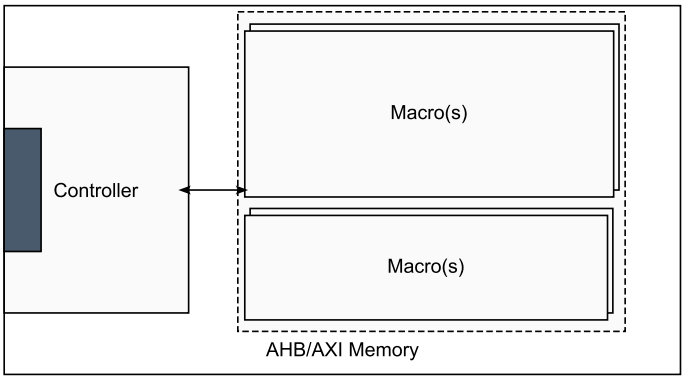



Figure 7.1: AXI/AHB SRAM module general block view.

## 7.1 Overview

SRAM macros usually expose a simple interface. Such an interface is seldom compatible with the port interfaces exposed by Network on Chip (NoC) or Network Interconnects (NIC). These systems employ elaborate on-chip interconnect standards like AHB/AXI[1] as they need to handle multiple transactions from multiple devices concurrently. Such protocols include transaction-related metadata signals which enable this capability. We require a controller frontend that will consume these highly informative AHB/AXI transactions and translate it into low-level SRAM interface access(es). We first provide a brief overview of the common controller logic for each of the protocol and then conclude with a description of the generator utility provided. Figure 7.1 shows the common block view of the SRAM module. The macros are instantiated independently of the controller via rule checking if using the optional configurator. Named `gen_lib_sram_*`, the user is expected to replace these prior to synthesis and/or pd with actual models from the foundry.

## 7.2 AHB Controller

The AHB family of macros uses `ahb2sram`, a 32-bit[2] AHB-Lite subordinate for a single-port synchronous SRAM. This controller module is parameterized with the geometry of the module the user expects. This can be different from the configuration of SRAM macros available to the user. The optional generator tool, discussed later, is capable to combining available macros to match the user's specific requirements.

Controller Parameters:

1. `LINE_W`: required SRAM line width in bits
2. `LINE_AW`: use to define the depth ($2^{\texttt{LINE\_AW}}$) of the SRAM in bits

[1] None of the AMBA Bus protocols will be discussed in detailed here. Please refer to the **official** documentation(s) available online.

[2] At the time of writing this, the controller only supports and is tested for 32-bit word size.

3. WORDS: number of 32-bit words. The current controller is only capable of supporting 32-bit words. Hence, this parameter should always equal LINE_W/32.

### 7.2.1 AHB-Lite Primer

An AHB-Lite transfer is split across two pipelined phases - address phase (aphase) and data phase (dphase). In the aphase the manager drives HADDR, HTRANS, HWRITE, and HSIZE, qualified by HSEL and the shared HREADY. In the subsequent dphase, write data (HWDATA) or read data (HRDATA) is exchanged, with the subordinate controlling the bus through its registered HREADYOUT and reporting status on HRESP. HTRANS[1] distinguishes an active (NONSEQ/SEQ) beat from IDLE/BUSY. Our implementation does not support burst-type transfers. This is in align with the behavior of the PicoRV32 core's native memory interface.

### 7.2.2 Phase Sequencing and Timing

The least significant bits of the address are word alignment bits and will be always zero for word size greater than 8 b. The next significant bit(s) are used for word select for wide SRAM lines. An active address beat is registered on aphase $=$ HSEL $\&$ HREADY $\&$ HTRANS[1]. A read drives the SRAM in the same cycle and its data is returned in the data phase; a write is held one cycle so that its data phase supplies HWDATA. Consequently a read incurs no wait state and a write incurs exactly one.

The chip enable, global write enable, and address to the macro are

$$\texttt{CEN} = \sim (\texttt{read\_en} \mid \texttt{write\_en\_reg}), \quad \texttt{GWEN} = \texttt{read\_en}, \quad \texttt{A} = \texttt{read\_en} \; ? \; \texttt{line\_addr\_nxt} : \texttt{line\_addr\_reg},$$

so a read presents the incoming address immediately while a write replays the registered one.

### 7.2.3 Byte Lanes and Write Enables

From HSIZE and the lower address bits the controller derives a one-hot-per-byte lane mask (byte, half-word, or full word), registers it into the data phase, and expands it into the macro's active-low, bitwise write enable. When a line is wider and can accomodate several bus-words (WORDS $> 1$) the addressed 32-bit word is selected within the line for both the write mask and the read data; otherwise the line is the word. On a read the registered byte lanes gate HRDATA; HRESP is tied to OKAY.

## 7.3 AXI Controller

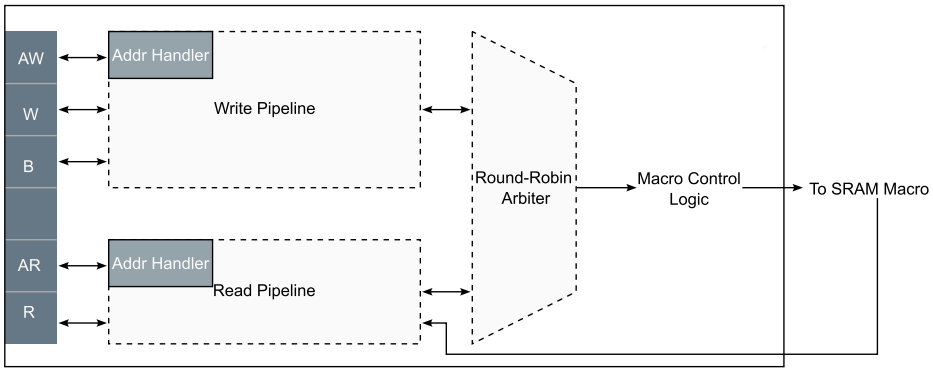



Figure 7.2: axi2sram controller detailed block view.

Similar to the AHB controller, there is an AXI controller (axi2sram) that converts AXI transactions to SRAM control logic. However, AXI is a more complex (and thereby capable) protocol compared to AHB. Figure Figure 7.2 provides an overview of the architecture of this controller. Note that the dotted lines are

only meant to represent sections of the logic and are not implemented as independent sub-modules. Only the address handler module that is shared by the both pipelines is implemented separately as a module.

### 7.3.1 AXI Addressing Primer

Most of the understanding about AXI addressing has to do with Burst transactions. Burst transactions involve multiple transfers. In these, the manager is expected to provide only a start address and the subordinate is expected to handle the rest of the transfers based on that address and some control signals.

There are two key control signals:

1. `AxSIZE`: This signal carries information about the number of bytes of information accessed in each transfer (also called a beat) within a Burst transaction. Specifically,

   $$\text{Number of bytes} = 2^{\texttt{AxSIZE}}.$$

   The width of this signal and the value it can assume is constrained by the data width. AXI doest not permit packet sizes greater than size of the Data bus. The width can be calculated as follows:

   $$\text{Width} = \$\texttt{clog2}\left(\frac{\texttt{DW}}{8}\right).$$

   ($`$clog2` rounds up; for a power-of-two `DW`/8 it is exact.)
2. `AxLEN [7:0]`: This indicates the number of transfers to be completed within a single Burst transaction. Specifically,

   $$\text{Number of transfers} = 1 + \texttt{AxLEN}$$

Here `Ax` stands for `AW` or `AR`.

There are three types of Burst transactions possible:

- FIXED: In a Burst type of FIXED type, same address is used repeatedly. An example application provided in the AXI specification is for emptying a FIFO. Generally, used very little.
- INCR: INCR Burst transactions are more common and align with the intuitive picture of a burst transfer. It sequential increments the address until specified.
- WRAP: A WRAP Burst transaction is similar to INCR, except it wraps around at a specific boundary depending on transaction attributes.

AXI "Regular" transactions impose certain restrictions on the attributes of the trasactions. See section A3.1.8 of AMBA IHI0022 document. Of those, our address generation submodule (`axi_addr_handler`) assumes the following:

1. Number of transfers: INCR bursts of $1$–$256$ beats; WRAP bursts of $\{2, 4, 8, 16\}$ beats.
2. Size is the same as the data channnel if Length is greater than 1.
3. Address is aligned to the transaction container for INCR transactions.
4. Address is aligned to Size for WRAP transactions.
5. Transactions are not permitted to cross a 4KB boundary (section A3.1.3, AMBA IHI0022)

### 7.3.2 Implementation

The AXI memory module consists of the following components: Main controller logic for handling AXI transactions, a submodule for handling burst addressing, and the native interfaced, SRAM macro. Here only discuss the controller. The SRAM macro exposes a read port (Q), write port (D), clock (CLK), chip enable port (CEN), global write enable (GWEN), local write enable (WEN), address port (A), and additional signals[3], STOV, EMA, EMAS, EMAW.

[3] These are ignored for simulation purposes.

1. Handling the addressing:
   The module `axi_addr_handler` is responsible for handling the addressing during Burst transactions. Given a start address, type of burst transfer, size of each transfer, and length of transaction, it generates addresses for each beat of the transaction. It is implemented as a two state FSM: IDLE and ACTIVE. It accepts new start address (called descriptor in the source code) whenever it is in IDLE state. This is indicated to the external environment by asserting the `start_addr_ready` signal. Once the external module asserts `start_addr_valid` signal, this module samples all the input signals, registers them, and enters ACTIVE state. Once in ACTIVE, it uses a count-down timer to track each beat. On the last count, the `is_last_addr` flag is asserted and the FSM moves into IDLE state. The module uses the `next_addr_ready/valid` handshake to move to the next address in the Burst transaction.
   The next address for Burst transactions is calculated as follows:

$$\texttt{next\_addr} = \begin{cases} \texttt{curr\_addr} & \text{if AxBURST = `FIXED} \\ \texttt{curr\_addr} + 2^{\texttt{AxSIZE}} & \text{if AxBURST = `INCR or within wrap boundary when AxBURST = `WRAP} \\ \texttt{wrap\_base} & \text{if AxBURST = `WRAP and crossing wrap boundary.} \end{cases} \tag{7.1}$$

   Here, `wrap_base` is the aligned base of the wrap region. Refer to the AXI specification for how this might be calculated[4]. This module assumes that the start address is aligned to Size and one increment indicates a word-size (element) increment of $2^{\texttt{AxSIZE}}$ bytes, i.e., the byte address will be incremented by 4 if Size = 4 bytes. Cache accesses is the main motivation of WRAP Bursts.
   AXI4 limits the length of WRAP transactions to 2/4/8/16. This provides a neat little trick used to calculate the next address in a Wrap transaction without using the modulo operator[5][6]. For these Lengths, the value of `AxLEN` is 0001, 0011, 0111, 1111, respectively. Notice that once the address has been adjusted for word alignment (i.e., those least significant zeros are shifted out), then only the bit positions where there is a 1 in value of `AxLEN` should be changed in the address (when `AxLEN`) is aligned with least significant end of the address. Rest of the address should carry over from the before. This trick is realized with the following equation:

$$\texttt{next\_addr\_wrap} = (\texttt{wrap\_mask} \mathbin{\&} \texttt{next\_addr\_incr}) \mid (\sim \texttt{wrap\_mask} \mathbin{\&} \texttt{curr\_addr}). \tag{7.2}$$

   In the above expression, most of the terms should be self-explanatory, except perhaps `wrap_mask`. This term is simply the `AxLEN` signal shifted to account for the word alignment and padded for address size. Bitwise complement of wrap mask has 1 in bit positions that should not change and be carried over from current address.
2. Main Controller Logic
   The main controller is implemented to try and mimic the behavior of `IntMemAxi` IP from ARM, which is a licensed IP that cannot be open-sourced. However, its functionality is described and can be found here. However, we cannot guarantee the identical behavior as our development was independent of this IP.
   This module is designed for controlling a single-port SRAM module. The reads and write paths are implemented independently and arbitrated in a round-robin fashion with reads being the default mode. Controller Parameters
   (a) `DW`: Word Size (only supports 32/64-bit)
   (b) `IDW`: AXI ID signals width
   (c) `LINE_W`: required SRAM line width
   (d) `LINE_AW`: use to define the depth of SRAM (in bits)
   (e) `WORDS`: Number of words per SRAM line
   (f) `READ_WAIT`: (0/1) Number of wait stages for reads.

[4] WRAP Burst is intended for cache implementations, where, to minimize stall penalty, the first word is delivered immediately and the the rest of the cache line is then fetched sequencially (wrapping).
[5] Credits for this trick: https://zipcpu.com/blog/2019/04/27/axi-addr.html.
[6] My instinct is that they limit the length for Wrap Bursts for this very reason. Or it may just be that 16 is sufficient for most applications.

This controller can support multiple words per SRAM line. Further, the reads can have an optional wait stage that allows pipelining. Note that like the AHB controller, the WORDS parameter should be sensible and will depend on the word size and the line width.
The write path is relatively straightforward. Whenever a new write request arrives, the address handler is used to unravel and stream the addresses in the burst sequence. Once available and the previous transaction is completed, a new request write request is raised.
The read path is slightly more complicated by the optional wait state. Whenever a new read request is granted by the arbiter, the read path becomes busy and the count down timer is topped up. Once the count down is complete, the read value appears on the read port of the macro and is subsequently transferred to `RDATA` bus when it is available.
Both, read and write, paths are pipelined to improve performance. Further, similar to AHB controller, rest of the logic is used to generate the line-wide masks by extending the Strobe signals and word select bits.

## 7.4 Libremc

Terminology: We use "module" refers to the entity that actually connects to a bus or a network on chip. The user specifies the module geometry. "Macro" is used to refer to the actual SRAM provided by the foundry/memory compilers. Such macros have a primitive interface and might have limited configurations.

We provide a support utility called Libremc (short for Libre macro compiler) that enables the SoC configurator to easily build SRAM modules for connecting to either Pico subsystem or the NoC Subsystem. Consequently, it can configure the

1. Frontend protocol: Either AXI or AHB.
2. Width (W): width of the SRAM module.
3. Depth (D): depth of the SRAM module.

The word size is fixed to 32 bits for AHB slave memories and to 64 bits for AXI ones[7]. Further, the address bus width is fixed to 32 bits.

However, often when synthesizing memory modules, one might not have access to the SRAM compilers or the available compilers might have limited capability and only a few macro configurations are accessible. Libremc allows the user to specify the available macro configurations in a `.conf` file as follows:

```
# Depth         Width   DepthStride  WidthStride  Mask-Granularity
16384           32      0            0            8
16384           64      0            0            8
4096            64      0            0            8
8192            64      0            0            8
512-8192        32      512          0            8
```

The values can be specified explicitly or as (low - high) ranges.

The tool uses this information to find the "optimal" combination to meet the user specified module requirements. To keep things simple, we solve a simple grid problem. For each available macro $m_i = (w_i, d_i)$ configuration, the following cost function $f_i$ is evaluated:

$$f_i = c_i \times r_i \times a_i, \tag{7.3}$$

[7]This is because the bus specifications are a part of the architecture which is common to the platform.

where, $c_i = \text{ceil}(W/w_i)$, $r_i = \text{ceil}(D/d_i)$, and $a_i$ is the area of macro. When unavailable, the area of the macro can be estimated as the product of a macro's width $w_i$ and depth $d_i$[8].

The tool selects the configuration as follows:

1. Configuration with the lowest value of cost function.
2. Any ties are broken by choosing the configuration with lowest number of instances ($r_i \times c_i$).
3. Remaining ties are broken by choosing the configuration using the smallest macro.

Once a configuration is found, the tool then automatically generates the module RTL along with a configuration file specifying the chosen macro and grid parameters. In this generated SRAM module it instantiates the requested controller and the grid of macros. In addition, it outputs the word select logic.

### 7.4.1 Usage

1. Generate a memory module:

```
$ make gen PROTO=<ahb|axi> SIZE=<lines>x<width> [MACROLIST=<file>.conf]
```

2. Elaborate and test a generated module:

```
$ make test-one PROTO=<ahb|axi> SIZE=<lines>x<width> [EXTRA_AW=n]
```

3. Generate and test a memory module in one step:

```
$ make test PROTO=<ahb|axi> SIZE=<lines>x<width> [MACROLIST=<file>] [EXTRA_AW=n]
```

4. Clean up generated files:

```
$ make clean
```

## 7.5 Summary

While the AHB controller logic was mostly inherited from Base-SoC (ARM), the AXI controller was developed in-house. AXI's Internal memory interface[9] behavior combined with SRAM Macro interface were used as a reference. We improved upon the IntMemAxi module[10] by directly supporting AXI4 (it supported AXI3) and directly generating the SRAM interface.

> Testing the Libremc utility is a non-trivial exercise and in fact a tedious one. Hence, we would appreciate any feedback on the tool, bug finds/fixes in the source code, and inaccuracies in this document.

[8] This does not capture the wiring overhead and may not correlate well with the actual area. We are working to include this wiring overhead to improve this tool.

[9] https://support.arm.com/documentation/dto0009/a/about-the-internal-memory-interface?lang=en

[10] We abide by the licensing terms by not using the source code for development of our own version. The functionality has been independently replicated.